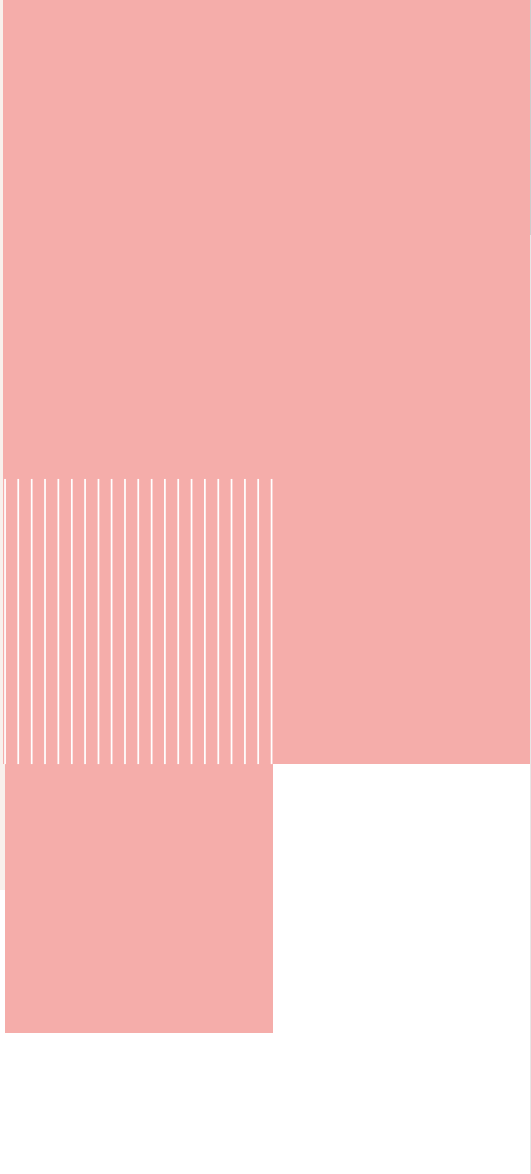


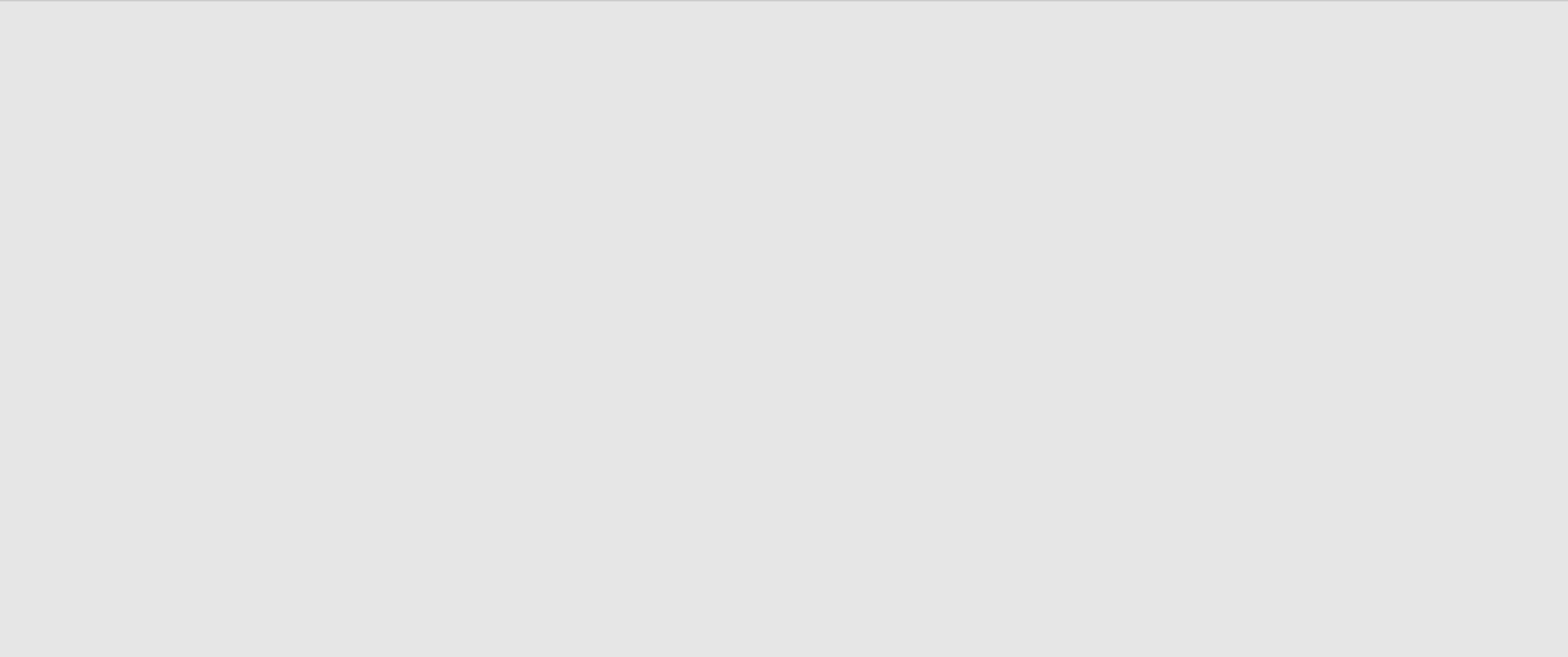
Soutenance de PFE

Maintien en condition de sécurité
de LF Energy SEAPATH





Contexte et enjeux



Le réseau électrique en transformation

- Consommation d'énergies renouvelables +65% en France en 11 ans [1]
 - + Besoin de **repenser gestion du réseau électrique** : *smart grids*

Contexte et enjeux

— Le réseau électrique en transformation

Le réseau électrique en transformation

- Consommation d'énergies renouvelables +65% en France en 11 ans [1]
 - + Besoin de **repenser gestion du réseau électrique** : *smart grids*



Fig. 1. – Photographie d'un **poste électrique**

- Nombreux équipements : disjoncteurs, transformateurs, etc.
- Contrôlés par des *Intelligent Electronic Devices* (IEDs)
 - + Défis des *smart grids* : besoin de **virtualiser** les IEDs

Contexte et enjeux

— Le réseau électrique en transformation

Place de SEAPATH [2]

Système d'exploitation :

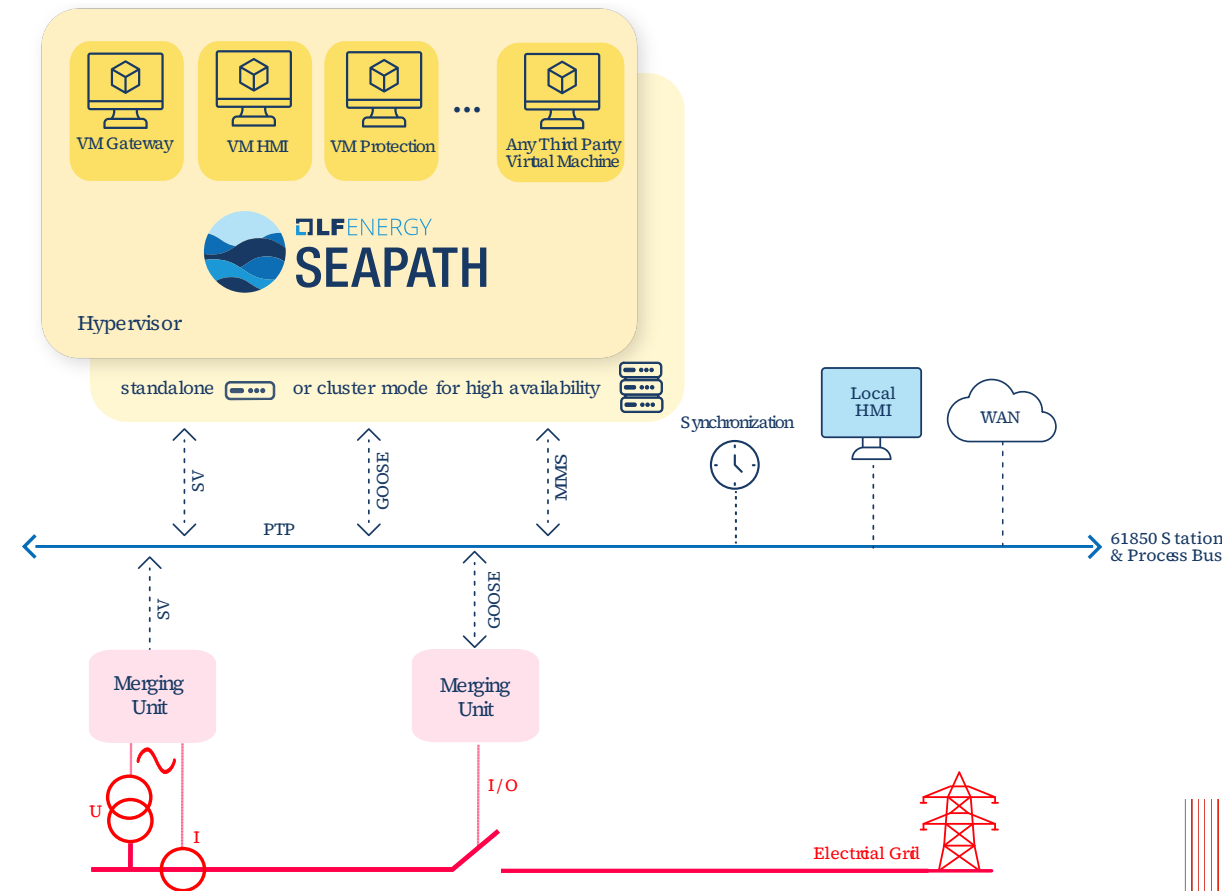
- pour VMs (**hyperviseur**)
- héberge des **IEDs virtuels**

Contraintes :

- **temps réel**
- **haute disponibilité**

Ouvert :

- **open source**
co-mainteneurs :
 - + Savoir-faire Linux
 - + RTE
- basé sur le noyau Linux



Contexte et enjeux — Place de SEAPATH [2]

Architecture de SEAPATH et enjeux

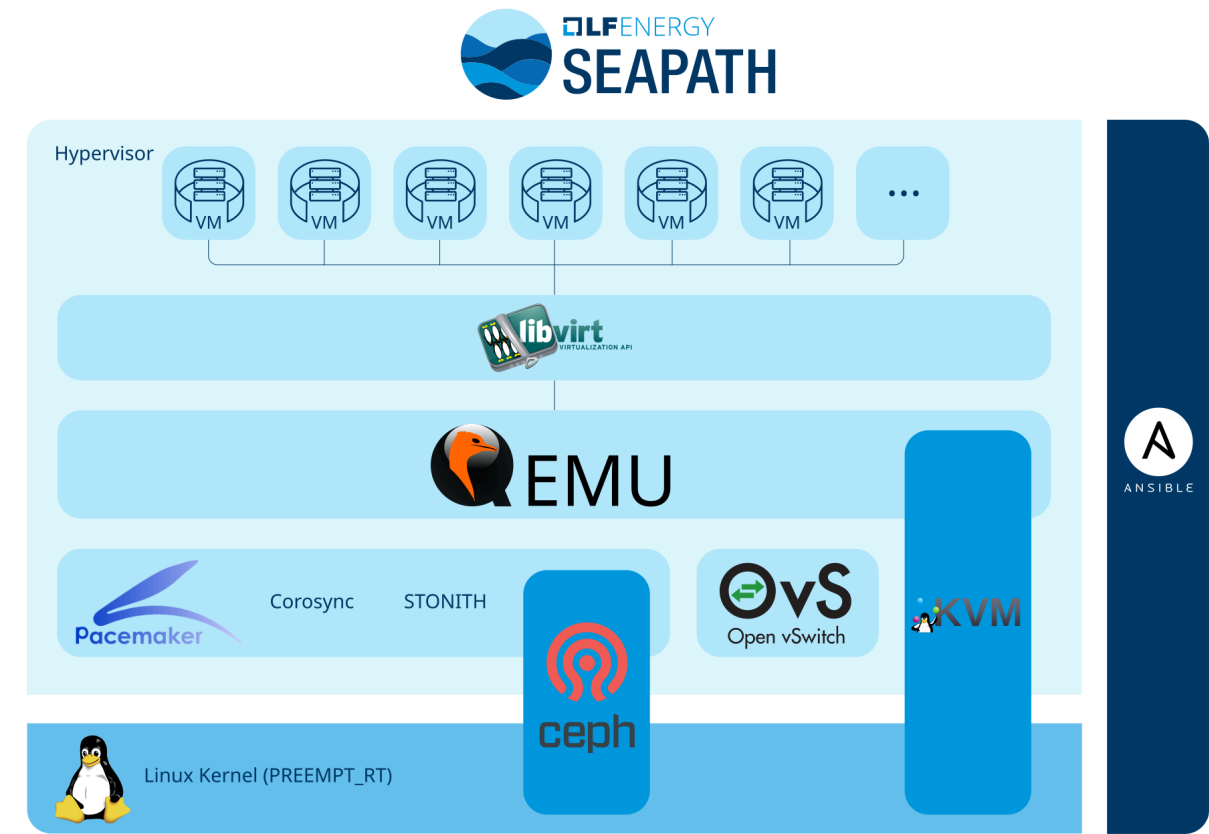


Fig. 3. – Architecture générale de SEAPATH

≈ Distribution Linux :
Assemblage de nombreux
composants logiciels.

Quelles vulnérabilités ?

SEAPATH destiné à des
infrastructures *critiques* !

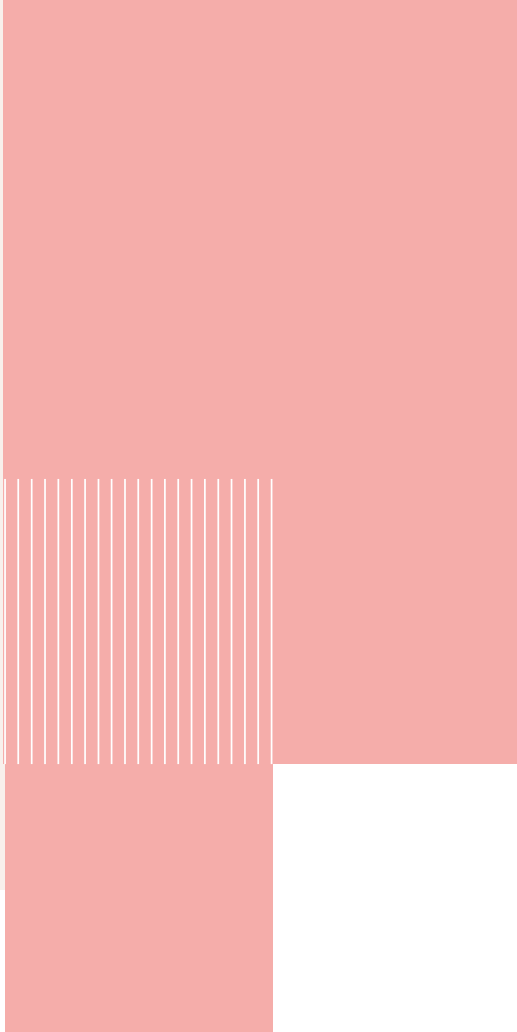
Législations : CRA (UE) /
Exec. Order 14028 (USA)

Contexte et enjeux
— Architecture de SEAPATH et enjeux

Composants **open source**



Présentation du stage



Présentation du stage

—

Missions

- **Recherche/comparaison** des **solutions de détection de vulnérabilités**
- **Mise en place** des solutions retenues
- **Analyser** les vulnérabilités remontées
 - + En utilisant et contribuant au projet **VulnScout** [3]

But recherché pour SEAPATH :

- Vue facile sur les vulnérabilités présentes dans SEAPATH
 - + Transparent pour les utilisateurs
 - + Mainteneurs peuvent potentiellement corriger en cas de vulnérabilité majeure



Présentation du stage

— Missions

Présenter rapidement VulnScout

Savoir-faire Linux

Entreprise de Services Numériques d'origine québécoise fondée en 1999.

Spécialité : technologies **libres**, systèmes et applications sous **Linux**.

Deux bureaux :

- **Montréal**, +50 collaborateurs, logiciels open source
- **Rennes**, ~15 collaborateurs (majorité d'ingénieurs en informatique), spécialisé Linux embarqué



Fig. 5. – Logo de l'entreprise



Fig. 6. – Jami [4], un logiciel développé par SFL

Méthodologie générale

Problème : SEAPATH $\approx$ distribution Linux : **centaines de composants**

$\Rightarrow$ **impossible** de détecter les failles dans le code (surface trop grande)

Présentation du stage
— Méthodologie générale

Méthodologie générale

Problème : SEAPATH $\approx$ distribution Linux : **centaines de composants**

$\Rightarrow$ **impossible** de détecter les failles dans le code (surface trop grande)

Notre approche : utiliser les **bases de données de vulnérabilités** (BDD)

- besoin de liste des composants présents = **SBOM**
- à faire générer avec SEAPATH

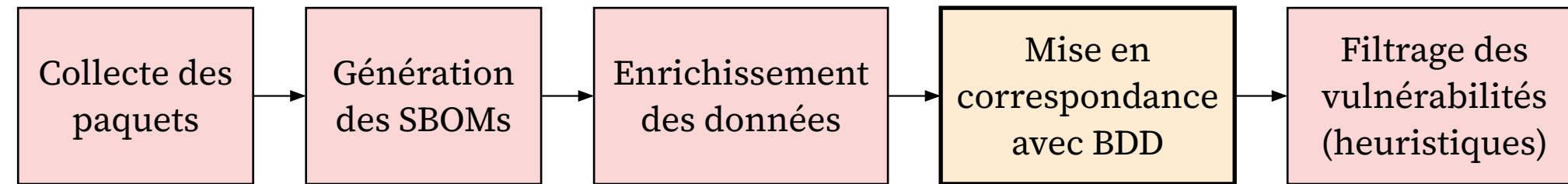


Fig. 7. – Processus de détection des vulnérabilités

Présentation du stage
— Méthodologie générale

Schéma haut niveau, on va voir chaque étape

BDD = base de données, évoquer exemples (NVD, CVE...)

Les SBOMs

- Contenu :
 - + **composants** : nom, versions, licences, etc.
 - + **fichiers**
 - + **relations** : contient / dépend de / décrit par / ...
- Interopérable, outils de **génération automatique**
- Formats standards : SPDX, CycloneDX



Fig. 8. – Standard ISO
(Linux Foundation)



Fig. 9. – Standard ecma
(OWASP)

Présentation du stage — Les SBOMs

Requis CRA

Dire que le contenu est très utile pour détecter vulnérabilités (identifiants composants)

Fichiers JSON trop verbeux pour être montré ici (ouverture)

Environnement technique : Yocto

- Projet open source, Fondation Linux
- Ensemble d'outils pour créer des **distributions Linux embarquées**



Construction du système avec BitBake :

- Décrit entièrement **via du code**
 - + tout est connu : programmes, bibliothèques, fichiers compilés...
 - + **facile de générer un SBOM**

Présentation du stage

— Environnement technique : Yocto

Parler de l'embarqué

Décrire «image»

On connaît les versions des programmes puisque c'est dans le code : nickel pour SBOM

Aucun binaire opaque inclus

Environnement technique : Yocto

- Projet open source, Fondation Linux
- Ensemble d'outils pour créer des **distributions Linux embarquées**



Construction du système avec BitBake :

- Décrit entièrement **via du code**
 - + tout est connu : programmes, bibliothèques, fichiers compilés...
 - + **facile de générer un SBOM**

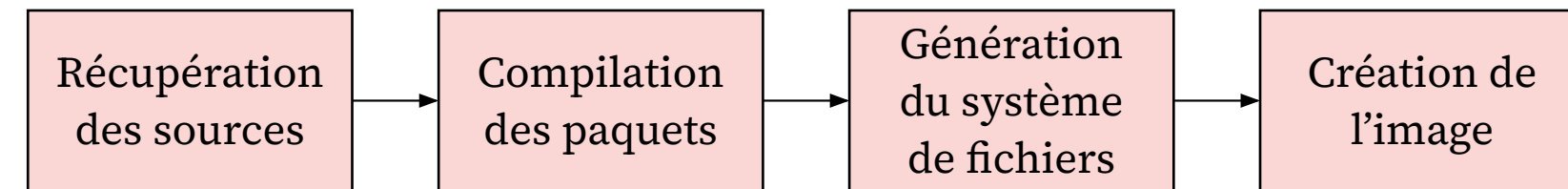


Fig. 11. – Processus de construction d'une image avec Yocto

Présentation du stage

— Environnement technique : Yocto

Parler de l'embarqué

Décrire «image»

On connaît les versions des programmes puisque c'est dans le code : nickel pour SBOM

Aucun binaire opaque inclus



Détection de vulnérabilités sur SEAPATH Yocto

Détection de vulnérabilités sur SEAPATH Yocto

—

Approches de détection de vulnérabilités

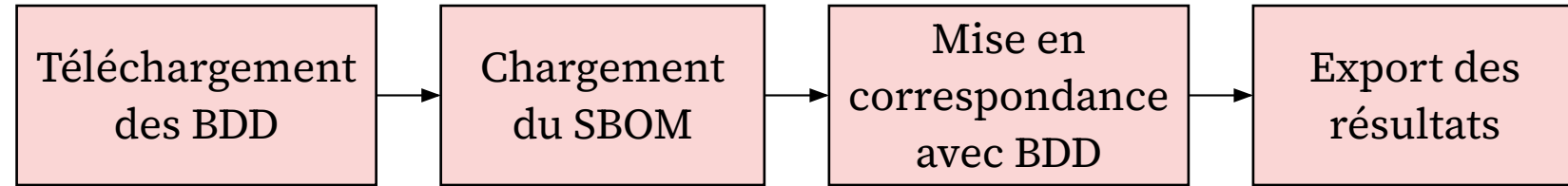
- Approches utilisant les SBOMs :
 - + **Grype** [5] : scanneur **orienté distributions** et conteneurs
 - + **sbom-cve-check** [6] : détecteur **orienté Yocto**



Détection de vulnérabilités sur SEAPATH Yocto
— Approches de détection de vulnérabilités

Approches de détection de vulnérabilités

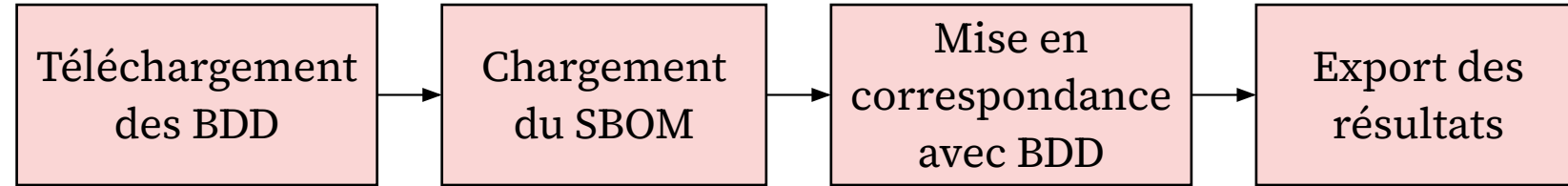
- Approches utilisant les SBOMs :
 - + **Grype** [5] : scanneur **orienté distributions** et conteneurs
 - + **sbom-cve-check** [6] : détecteur **orienté Yocto**



Détection de vulnérabilités sur SEAPATH Yocto
— Approches de détection de vulnérabilités

Approches de détection de vulnérabilités

- Approches utilisant les SBOMs :
 - + **Grype** [5] : scanneur **orienté distributions** et conteneurs
 - + **sbom-cve-check** [6] : détecteur **orienté Yocto**

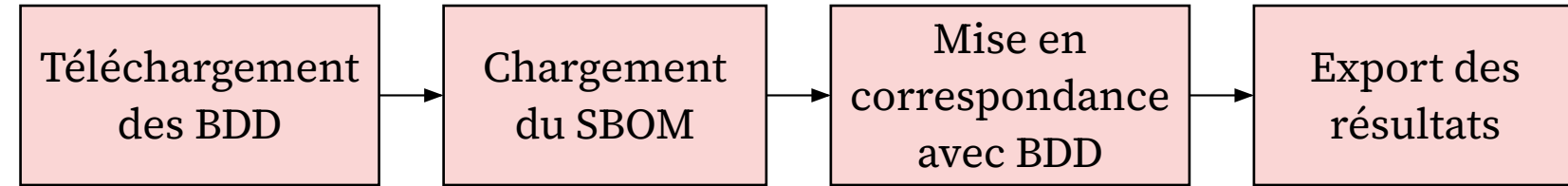


- Approche sans SBOMs : **cve-check**, classe pré-faite de Yocto
 - + cherche les vulnérabilités **lors de la construction** de l'image

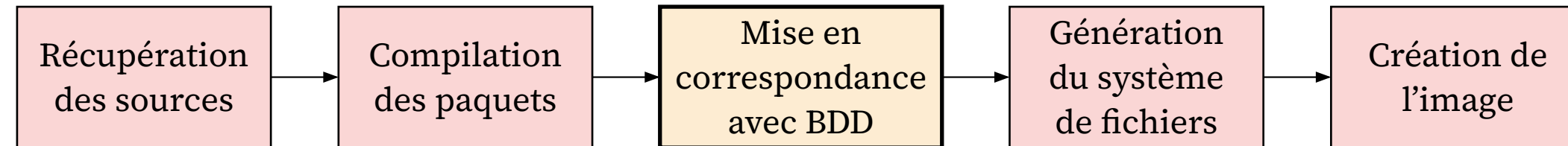
Détection de vulnérabilités sur SEAPATH Yocto
— Approches de détection de vulnérabilités

Approches de détection de vulnérabilités

- Approches utilisant les SBOMs :
 - + **Grype** [5] : scanneur **orienté distributions** et conteneurs
 - + **sbom-cve-check** [6] : détecteur **orienté Yocto**



- Approche sans SBOMs : **cve-check**, classe pré-faite de Yocto
 - + cherche les vulnérabilités **lors de la construction** de l'image



Détection de vulnérabilités sur SEAPATH Yocto — Approches de détection de vulnérabilités

Différences entre les approches

Base de donnée \ approche	NVD [7] (NIST)	CVE Program [8]	Noyau Linux [9]	Distributions (Debian, RedHat...)
cve-check	✓	✗	✗	✗
sbom-cve-check	✓	✓	✗	✗
Grype	✓	✗	✗	✓

cve-check **très limité** (NVD incomplète)

Différences entre les approches

Base de donnée \ approche	NVD [7] (NIST)	CVE Program [8]	Noyau Linux [9]	Distributions (Debian, RedHat...)
cve-check	✓	✗	✗	✗
↑ meta-vulnscout	✓	✓	✓	✗
sbom-cve-check	✓	✓	✗	✗
Grype	✓	✗	✗	✓

cve-check très limité (NVD incomplète)

⇒ meta-vulnscout

- rajoute 2 bases (CVE et noyau Linux)
- filtre vulnérabilités du noyau non applicables

Détection de vulnérabilités sur SEAPATH Yocto
— Différences entre les approches

Évaluation des approches

Critères :

- **nombre total** de vulnérabilités trouvées
- taux de **faux positifs / faux négatifs**
- facilité d'utilisation / d'automatisation

4 scénarios ⇒ comparaison des résultats :

1. **basique** : *cve-check*
2. **léger** : *cve-check* + BDDs supplémentaires
3. **complet** : *cve-check* + BDDs supplémentaires + filtrage noyau (*meta-vulnscout*)
4. **externe** : SBOM → *sbom-cve-check*

Vuln. trouvée	Vuln. applicable	
✓	✗	Faux positif
✗	✓	Faux négatif

Détection de vulnérabilités sur SEAPATH Yocto
— Évaluation des approches

Intentionnellement laissé Grype de côté

Résultats de la comparaison

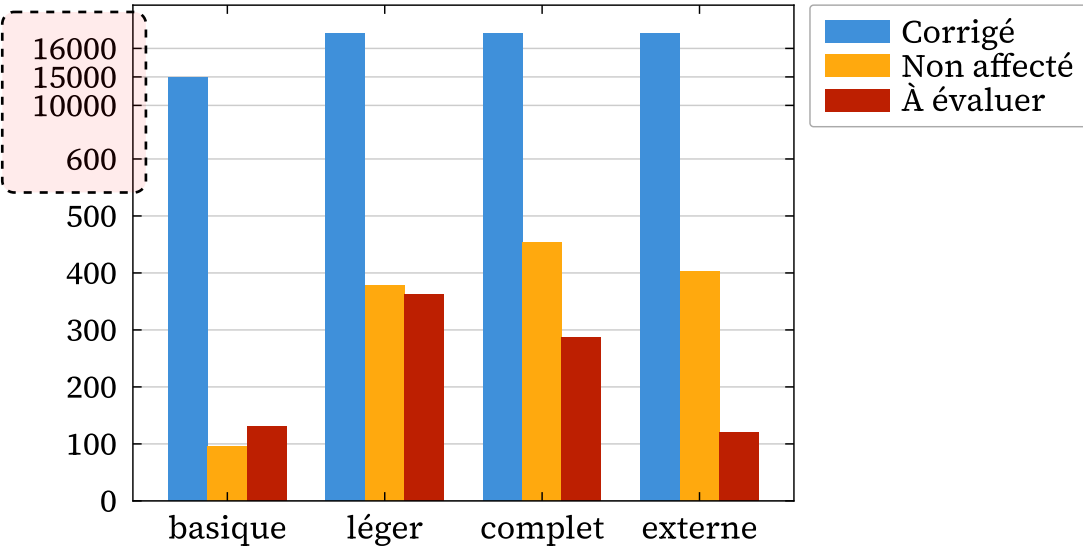


Fig. 17. – Vulnérabilités détectées par scénario

- Beaucoup de *non affecté* (jaune) = mieux
- Peu de *à évaluer* (rouge) = mieux

Principale différence = filtre sur **vulnérabilités noyau** :

- -20% léger $\Rightarrow$ complet
- -58% complet $\Rightarrow$ externe

Détection de vulnérabilités sur SEAPATH Yocto — Résultats de la comparaison

Attention axe Y

Résultats de la comparaison

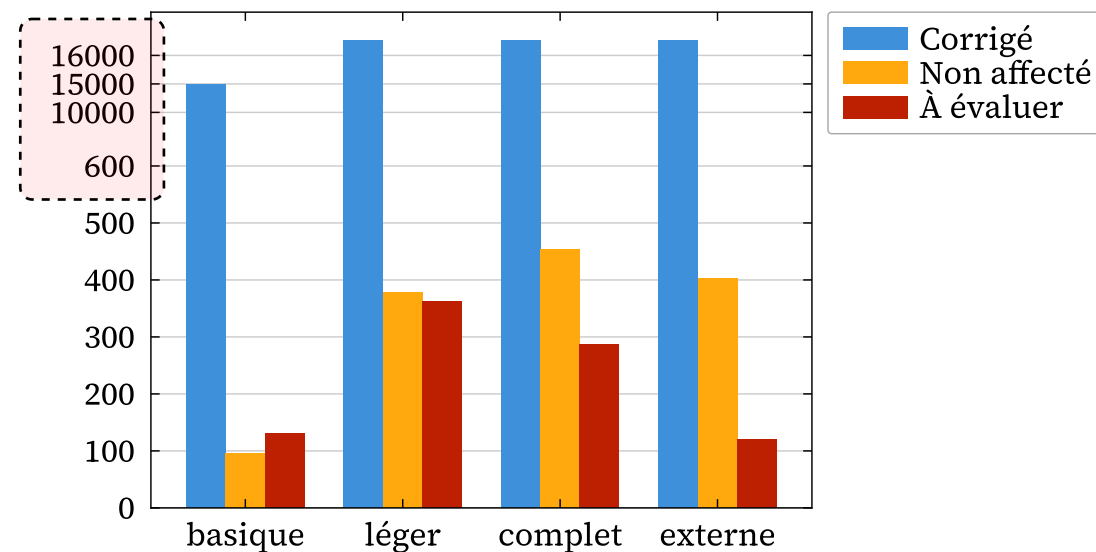


Fig. 19. – Vulnérabilités détectées par scénario

- Beaucoup de *non affecté* (jaune) = mieux
- Peu de *à évaluer* (rouge) = mieux

Principale différence = filtre sur
vulnérabilités noyau :

- -20% léger $\Rightarrow$ complet
- -58% complet $\Rightarrow$ externe

Résultats :

- *cve-check* basique : peu performant
+ beaucoup de faux négatifs
- *cve-check* + meta-vulnscout : mieux
+ faux positifs
- ***sbom-cve-check* : le meilleur**
+ facile d'utilisation

$\Rightarrow$ tout détaillé dans un **rapport**

Détection de vulnérabilités sur SEAPATH Yocto — Résultats de la comparaison

Attention axe Y

sbom-cve-check utilisable en CI car hors de Yocto

Rapport :

- utile pour chiffrer offres client
- estimer le nombre de vulns à évaluer

Analyse des vulnérabilités remontées

	ID	Severity	EPSS Score	SBOM Affected	Variants	Actions
	CVE-2026-33845	critical	0.07%	gnutls@3.8.12 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-5450	critical	0.07%	glibc@2.43 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-6100	critical	0.16%	python3@3.14.4 (unknown supplier)	guest_efi host_standalone_efi	Edit
Total: 50						

Fig. 20. – Interface de VulnScout

- ~130 vulnérabilités détectées
- filtre sur les plus importantes avec seuils :

$$CVSS^1 \geq 9.0$$

ou

$$CVSS \geq 7.0 \wedge EPSS^2 \geq 50\%$$

¹CVSS : **gravité** d’une vulnérabilité, de 0 à 10

²EPSS : **exploitabilité** d’une vulnérabilité, de 0 à 100%

Détection de vulnérabilités sur SEAPATH Yocto — Analyse des vulnérabilités remontées

Présenter VulnScout ici : pas d’outil open source pour évaluer CVEs avant

Choix fait avec tuteur + concertation avec qqun qui s’y connaît + sera ré-étudié lors du comité de pilotage technique (R&D pour le moment)

Analyse des vulnérabilités remontées

	ID	Severity	EPSS Score	SBOM Affected	Variants	Actions
	CVE-2026-33845	critical	0.07%	gnutls@3.8.12 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-5450	critical	0.07%	glibc@2.43 (unknown supplier)	guest_efi host_standalone_efi	Edit
	CVE-2026-6100	critical	0.16%	python3@3.14.4 (unknown supplier)	guest_efi host_standalone_efi	Edit
Total: 50						

Fig. 29. – Interface de VulnScout

- ~130 vulnérabilités détectées
- filtre sur les plus importantes avec seuils :

$$CVSS^1 \geq 9.0$$

ou

$$CVSS \geq 7.0 \wedge EPSS^2 \geq 50\%$$

- Reste ~15 vulnérabilités : majorité **faux positifs**

¹CVSS : **gravité** d’une vulnérabilité, de 0 à 10

²EPSS : **exploitabilité** d’une vulnérabilité, de 0 à 100%

Détection de vulnérabilités sur SEAPATH Yocto — Analyse des vulnérabilités remontées

Présenter VulnScout ici : pas d’outil open source pour évaluer CVEs avant

Choix fait avec tuteur + concertation avec qqun qui s’y connaît + sera ré-étudié lors du comité de pilotage technique (R&D pour le moment)

EPSS: machine learning, proba exploité dans les 30 jours.
data: âge de la vuln, listée sur des sites d’exploit, code d’exploit dispo, présent dans outils de sécu offensive...

Correction des faux positifs

BitBake

```
# recipes-connectivity/inetutils
CVE_STATUS[CVE-2026-32746] = "not-applicable-
config: telnetd not included in SEAPATH"

# recipes-extended/libvirt
CVE_STATUS[CVE-2018-6764] = "fixed-version: Fixed
in 4.1.0, NVD tracks this as version-less
vulnerability"
# ...et plein d'autres
```

Fig. 30. – Annotations de vulnérabilités

Détection de vulnérabilités sur SEAPATH Yocto — Correction des faux positifs

Correction des faux positifs

BitBake

```
# recipes-connectivity/inetutils
CVE_STATUS[CVE-2026-32746] = "not-applicable-
config: telnetd not included in SEAPATH"

# recipes-extended/libvirt
CVE_STATUS[CVE-2018-6764] = "fixed-version: Fixed
in 4.1.0, NVD tracks this as version-less
vulnerability"
# ...et plein d'autres
```

Fig. 30. – Annotations de vulnérabilités

CVE-2026-32746

Severity: critical

EPSS Score: 5.30%

Found by: SPDX3

Status: Not affected

Affects: inetutils@2.7 (unknown supplier)

Aliases:

Related vulnerabilities:

CVSS

CVSS 3.1

9.8

cve@mitre.org

Description

telnetd in GNU inetutils through 2.7 allows an out-of-bounds write in the LINEMODE SLC (Set Local Characters) suboption handler because add_slc does not check whether the buffer is full.

Links

<https://www.cve.org/CVERecord?id=CVE-2026-32746>

Assessments

2 juin 2026 à 14:55 UTC+2

inetutils@2.7 (unknown supplier)

guest_efi host_standalone_efi

Not affected - vulnerable_code_not_present

telnetd not included in SEAPATH

Fig. 31. – Vulnérabilité sur VulnScout

Détection de vulnérabilités sur SEAPATH Yocto — Correction des faux positifs

Correction des faux positifs

BitBake

```
# recipes-connectivity/inetutils
CVE_STATUS[CVE-2026-32746] = "not-applicable-
config: telnetd not included in SEAPATH"

# recipes-extended/libvirt
CVE_STATUS[CVE-2018-6764] = "fixed-version: Fixed
in 4.1.0, NVD tracks this as version-less
vulnerability"
# ...et plein d'autres
```

Fig. 30. – Annotations de vulnérabilités

⇒ Annotations type fixed-version
contribuées au projet Yocto

CVE-2026-32746

Severity: critical

EPSS Score: 5.30%

Found by: SPDX3

Status: Not affected

Affects: inetutils@2.7 (unknown supplier)

Aliases:

Related vulnerabilities:

CVSS

CVSS 3.1

9.8

cve@mitre.org

Description

telnetd in GNU inetutils through 2.7 allows an out-of-bounds write in the LINEMODE SLC (Set Local Characters) suboption handler because add_slc does not check whether the buffer is full.

Links

<https://www.cve.org/CVERecord?id=CVE-2026-32746>

Assessments

2 juin 2026 à 14:55 UTC+2

inetutils@2.7 (unknown supplier)

guest_efi host_standalone_efi

Not affected - vulnerable_code_not_present

telnetd not included in SEAPATH

Fig. 31. – Vulnérabilité sur VulnScout

Détection de vulnérabilités sur SEAPATH Yocto — Correction des faux positifs

Automatisation via Intégration Continue (CI)

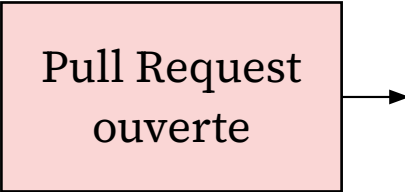


Fig. 32. – Processus de la CI pour une Pull Request

Automatisation via Intégration Continue (CI)

1. Ré-écriture du pipeline de construction sur GitHub Actions [10]

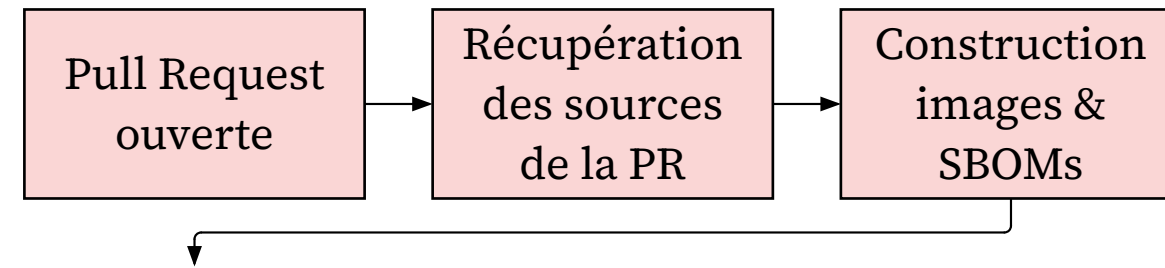


Fig. 33. – Processus de la CI pour une Pull Request

Détection de vulnérabilités sur SEAPATH Yocto
— Automatisation via Intégration Continue (CI)

Évoquer synchronisation entre dépôts

Automatisation via Intégration Continue (CI)

1. Ré-écriture du pipeline de construction sur GitHub Actions [10]
2. Détection des vulnérabilités
 - avec *sbom-cve-check*

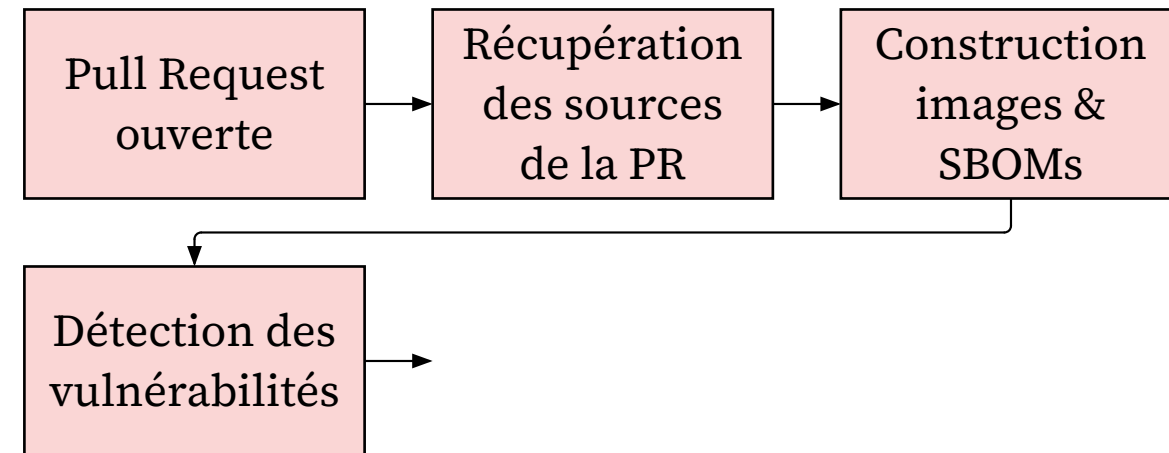


Fig. 33. – Processus de la CI pour une Pull Request

Détection de vulnérabilités sur SEAPATH Yocto
— Automatisation via Intégration Continue (CI)

Évoquer synchronisation entre dépôts

Automatisation via Intégration Continue (CI)

1. Ré-écriture du pipeline de construction sur GitHub Actions [10]
2. Détection des vulnérabilités
 - avec *sbom-cve-check*
3. Génération de rapports affichés dans les Pull Requests
 - avec *VulnScout*

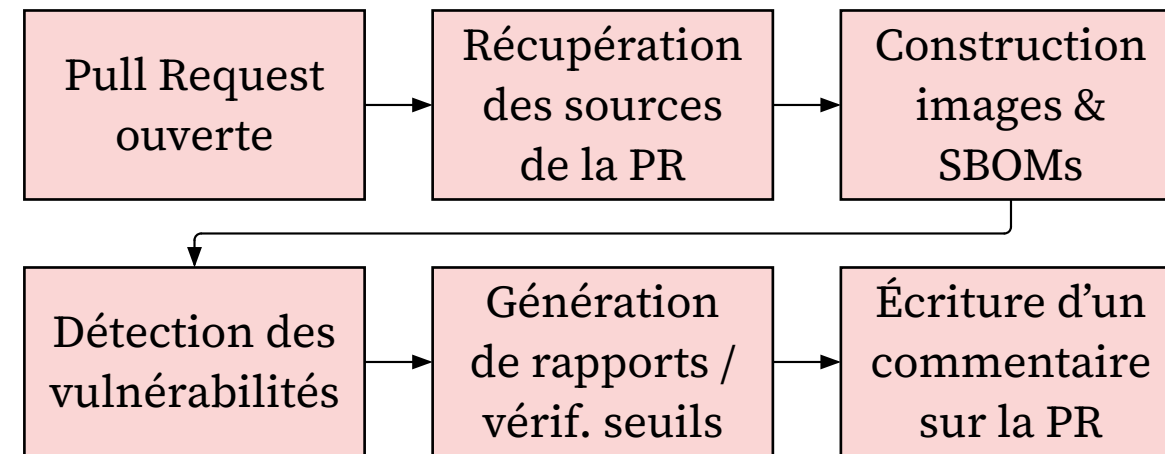


Fig. 33. – Processus de la CI pour une Pull Request

Détection de vulnérabilités sur SEAPATH Yocto — Automatisation via Intégration Continue (CI)

Évoquer synchronisation entre dépôts

Intégration Continue : Rapports et commentaires

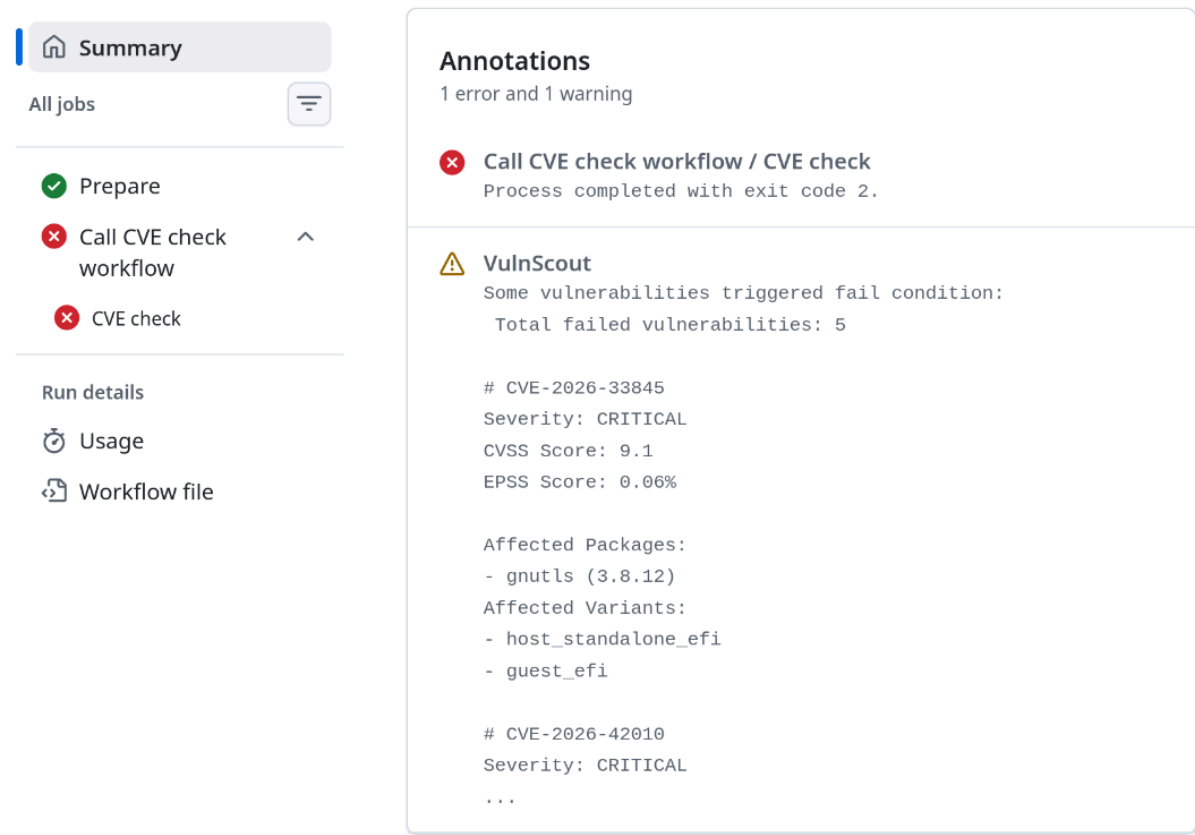


Fig. 34. – Vulnérabilités dépassant les seuils affichées dans le résumé de la CI

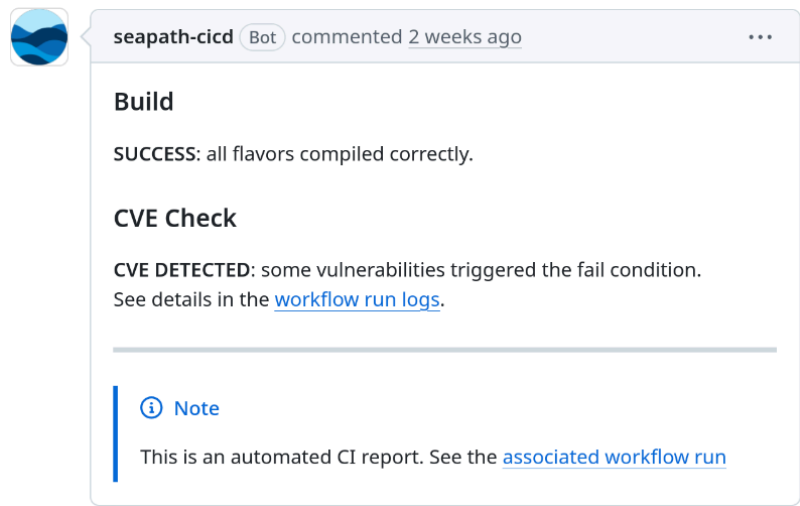


Fig. 35. – Commentaire automatique sur la Pull Request

Détection de vulnérabilités sur SEAPATH Yocto – Intégration Continue : Rapports et commentaires



Détection de vulnérabilités sur SEAPATH Debian

Détection de vulnérabilités sur SEAPATH Debian

—

- Autre version de SEAPATH basée sur la distribution Debian [11]
- Projet FAI [12] pour automatiser image disque :
 - + liste de paquets
 - + configuration
 - + scripts



debian

- Autre version de SEAPATH basée sur la distribution Debian [11]
- Projet FAI [12] pour automatiser image disque :
 - + liste de paquets
 - + configuration
 - + scripts

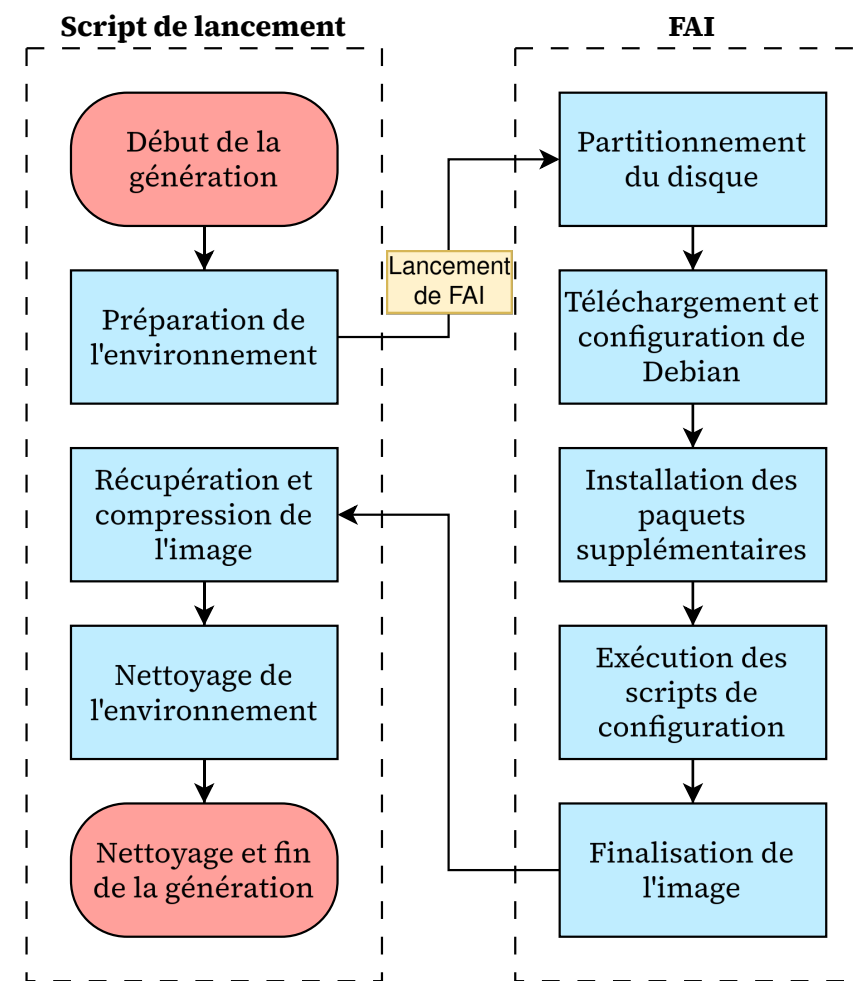


Fig. 38. – Génération de la version Debian

Méthodologie de détection

- Pas construit depuis les sources $\Rightarrow$ autre approche pour SBOM
 - + **scanner le système de fichiers** pour trouver programmes, librairies, etc. : utilisation de **Syft**
 - + À intégrer dans le processus FAI



Détection de vulnérabilités sur SEAPATH Debian
— Méthodologie de détection

Méthodologie de détection

- Pas construit depuis les sources $\Rightarrow$ autre approche pour SBOM
 - + **scanner le système de fichiers** pour trouver programmes, librairies, etc. : utilisation de **Syft**
 - + À intégrer dans le processus FAI
- **Détection des vulnérabilités** depuis le SBOM avec **Grype**



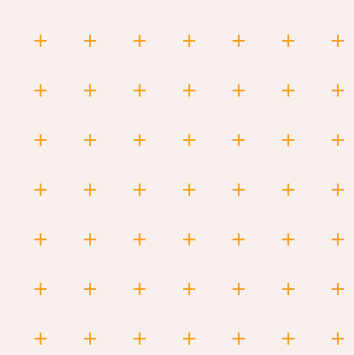
Détection de vulnérabilités sur SEAPATH Debian
— Méthodologie de détection

Méthodologie de détection

- Pas construit depuis les sources $\Rightarrow$ autre approche pour SBOM
 - + **scanner le système de fichiers** pour trouver programmes, librairies, etc. : utilisation de **Syft**
 - + À intégrer dans le processus FAI
- **Détection des vulnérabilités** depuis le SBOM avec **Grype**
- **Intégration en CI**
 - + **rappports** (comme version Yocto)



Détection de vulnérabilités sur SEAPATH Debian
— Méthodologie de détection



VulnScout

VulnScout

Contexte du projet

- Créé en mai 2024 (PFE d'un étudiant de l'INSA CVL)
- Open source, maintenu par une équipe de Savoir-faire Linux Montréal
- Application web :
 - + Front-end TypeScript / React [13]
 - + Back-end Python / Flask [14]
 - + Données stockées dans une BDD relationnelle

Utilisation et amélioration pour SEAPATH

- Évaluation des vulnérabilités présentes
- En CI : génération de rapports, test des seuils

⇒ Contributions :

- **Correction de bugs**
- Ajout de **fonctionnalités manquantes**, e.g. :
 - + Commandes pour réinitialiser la BDD dans la CI
 - + Chargement de champs SPDX supplémentaires

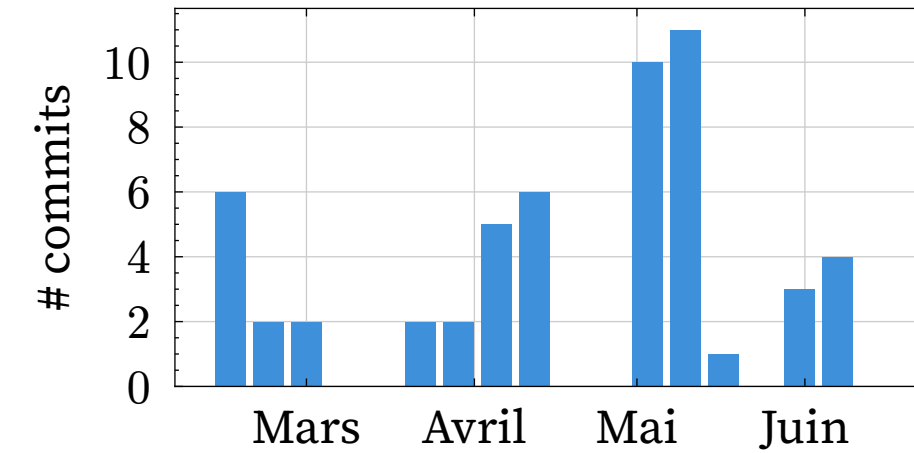


Fig. 45. – Contributions à VulnScout par semaine

VulnScout

— Utilisation et amélioration pour SEAPATH

Amélioration de la qualité de code

Pas de typage dans le backend = difficile de faire évoluer le code

Pratiques mises en place :

- nouveau code écrit « propre »
- **revue des Pull Requests** avec attention particulière

VulnScout

— Amélioration de la qualité de code

Dette technique

Pression d'ajout de nouvelles fonctionnalités

Peu de temps accordé à l'amélioration de l'ancien code dans les sprints

Amélioration de la qualité de code

Pas de typage dans le backend = difficile de faire évoluer le code

Pratiques mises en place :

- nouveau code écrit « propre »
- **revue des Pull Requests** avec attention particulière

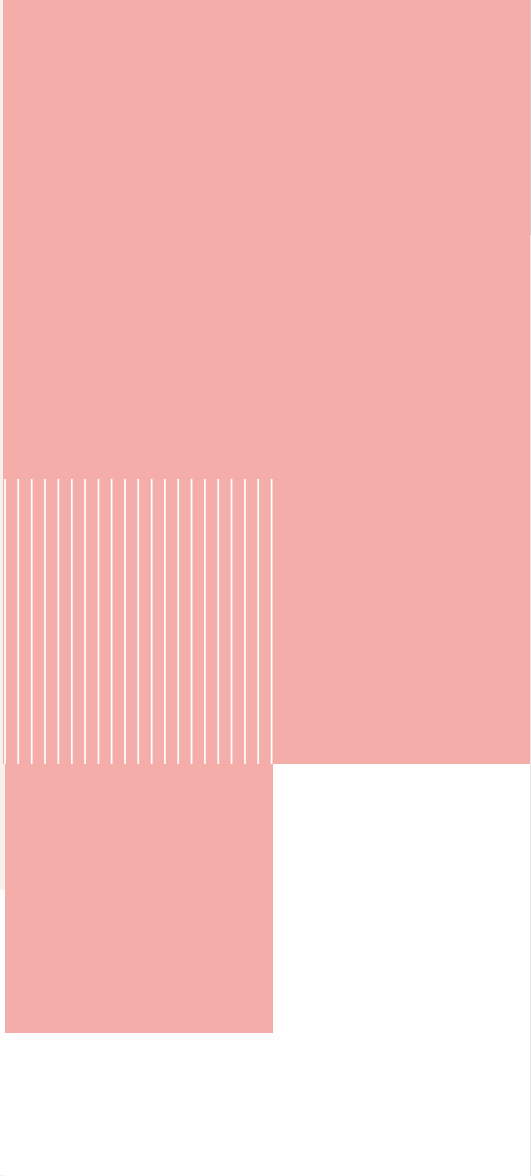
Version	Lignes de code	mypy	Pyright	
		Imprécision	Exhaustivité du typage	Symboles avec types connus
0.10	6214	37,51 %	32,9 %	156 / 430
0.17-a	20391	30,56%	45,3 %	469 / 997

Tableau 4. – Évolution des indicateurs de qualité du typage du backend au cours du stage

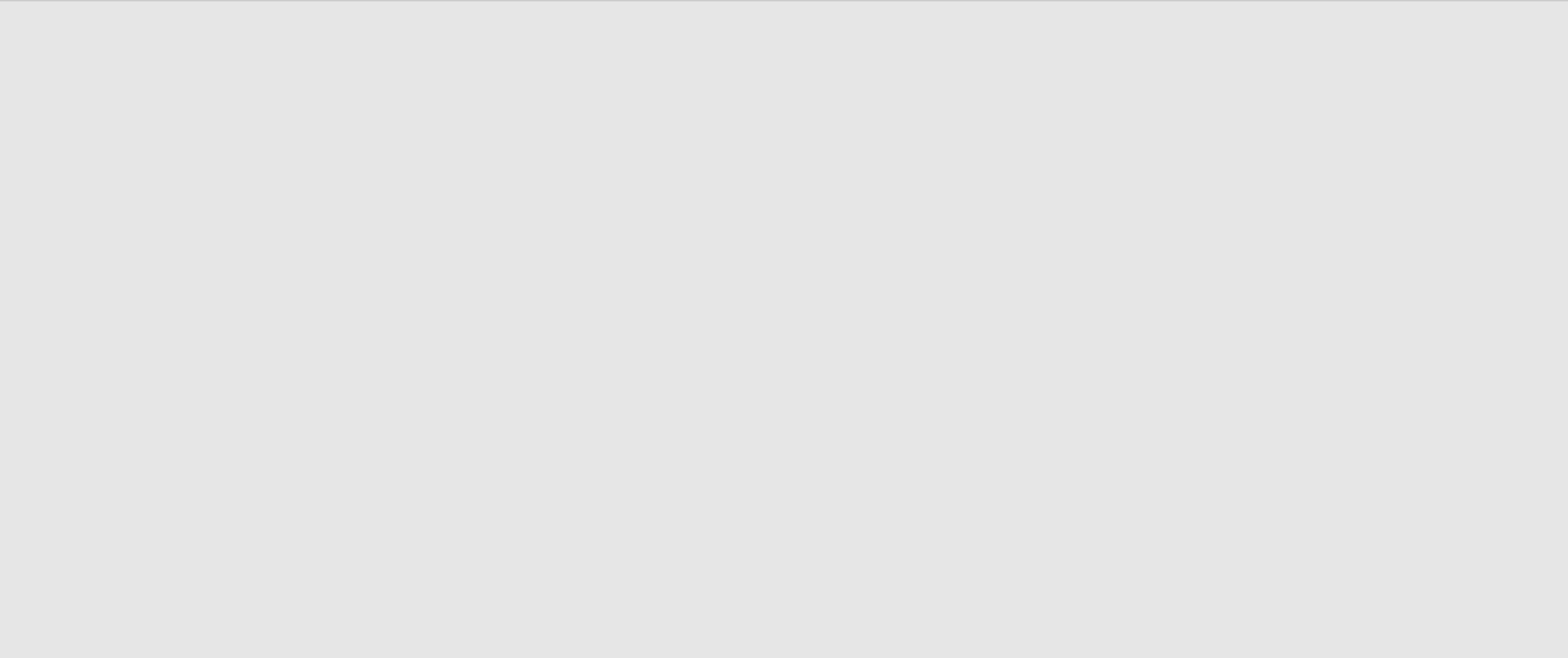
Dette technique

Pression d’ajout de nouvelles fonctionnalités

Peu de temps accordé à l’amélioration de l’ancien code dans les sprints



Conclusion



- SEAPATH Yocto :
 - + **Étude et rapport** sur approches de **détection de vulnérabilités**
 - + **Implémentation** de la meilleure solution
 - + **Ré-écriture de l'intégration continue** dans GitHub Actions
 - + **Automatisation** de la détection

Conclusion

—

- SEAPATH Yocto :
 - + **Étude et rapport** sur approches de **détection de vulnérabilités**
 - + **Implémentation** de la meilleure solution
 - + **Ré-écriture de l'intégration continue** dans GitHub Actions
 - + **Automatisation** de la détection
- SEAPATH Debian :
 - + **Implémentation** d'une solution de détection de vulnérabilités
 - + Ajout dans l'**intégration continue**
- *À faire* : seuils, décider quoi faire des vulnérabilités détectées

Conclusion

—

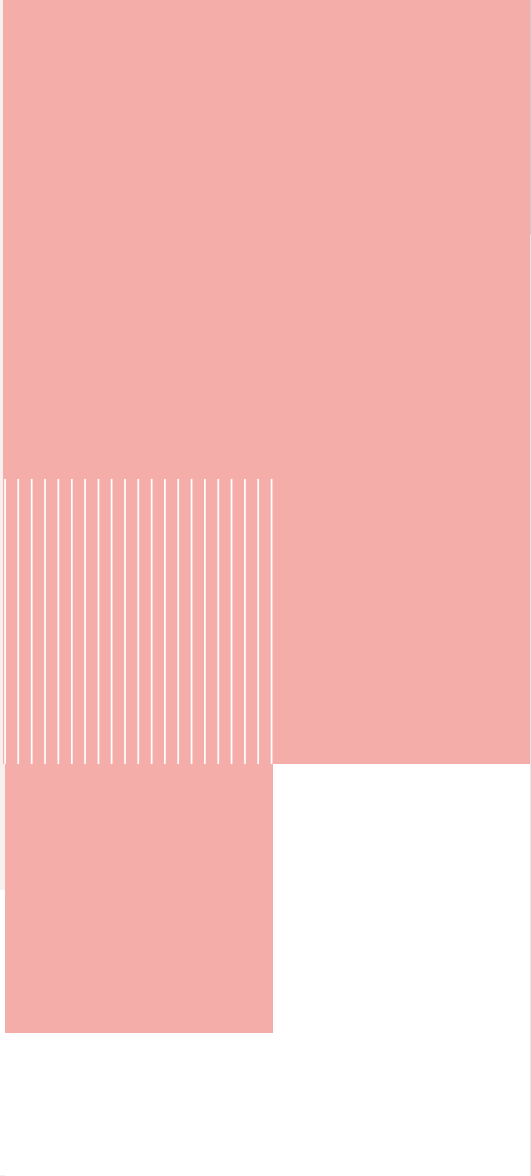
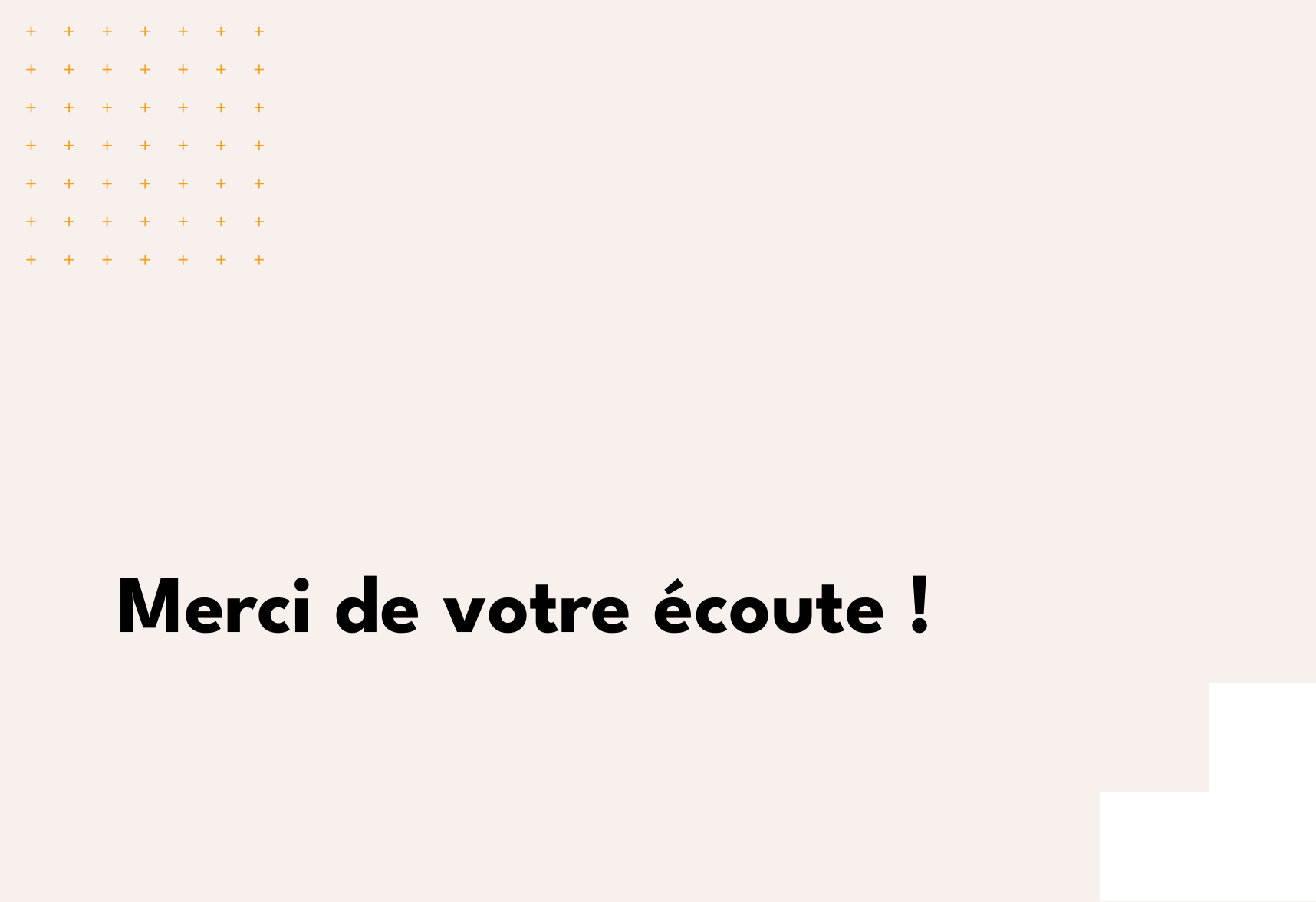
- SEAPATH Yocto :
 - + **Étude et rapport** sur approches de **détection de vulnérabilités**
 - + **Implémentation** de la meilleure solution
 - + **Ré-écriture de l'intégration continue** dans GitHub Actions
 - + **Automatisation** de la détection
- SEAPATH Debian :
 - + **Implémentation** d'une solution de détection de vulnérabilités
 - + Ajout dans l'**intégration continue**
- À *faire* : seuils, décider quoi faire des vulnérabilités détectées
- VulnScout :
 - + **Contributions** pour résoudre problèmes lors de l'utilisation
 - + Amélioration de la **qualité du code**

Conclusion

—

Plan personnel :

- travailler sur projets open source au sein d'une entreprise = bonne expérience
- opportunité de contribuer à un grand projet open source (Yocto)
- intégré à une équipe de haute compétence



Conclusion

—

Bibliographie

[1] SDES, « Chiffres clés des énergies renouvelables », 2025. Consulté le: 13 mai 2026. [En ligne]. Disponible sur: <https://www.statistiques.developpement-durable.gouv.fr/edition-numerique/chiffres-cles-energies-renouvelables/fr/>

[2] LF Energy, « SEAPATH ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://lfenergy.org/projects/seapath/>

[3] « VulnScout: The Easy-to-Use Tool for Securing Embedded Systems and Staying Compliant », Savoir-faire Linux. Consulté le: 23 avril 2026. [En ligne]. Disponible sur: <https://blog.savoirfairelinux.com/en-ca/2025/vulnscout-the-easy-to-use-tool-for-securing-embedded-systems-and-staying-compliant/>

[4] « Jami ». Consulté le: 18 mai 2026. [En ligne]. Disponible sur: <https://jami.net/>

[5] Trex Team, *Grype in Production*. NobleTrex Press, 2026. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: https://books.google.com/books/about/Grype_in_Production.html?hl=fr&id=LJ_KEQAAQBAJ

[6] bootlin, *sbom-cve-check*. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://github.com/bootlin/sbom-cve-check/>

Bibliographie

—

- [7] H. Booth, « National Vulnerability Database ». Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://nvd.nist.gov/>
- [8] CVEProject, *cvelistV5*. Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://github.com/CVEProject/cvelistV5>
- [9] Linux, *vulns*. Consulté le: 11 mars 2026. [En ligne]. Disponible sur: <https://git.kernel.org/pub/scm/linux/security/vulns.git/>
- [10] C. Chandrasekara et P. Herath, *Hands-on GitHub Actions: Implement CI/CD with GitHub Action Workflows for Your Applications*. Berkeley, CA: Apress, 2021. doi: 10.1007/978-1-4842-6464-5.
- [11] The Debian Project, « Debian Reference ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://www.debian.org/doc/manuals/debian-reference/>
- [12] Thomas Lange, « FAI - Fully Automatic Installation ». Consulté le: 4 juin 2026. [En ligne]. Disponible sur: <https://fai-project.org/>
- [13] Facebook, *React*. Consulté le: 3 juin 2026. [En ligne]. Disponible sur: <https://github.com/facebook/react>
- [14] Pallets, « Flask Documentation ». Consulté le: 3 juin 2026. [En ligne]. Disponible sur: <https://flask.palletsprojects.com/en/stable/>

Bibliographie

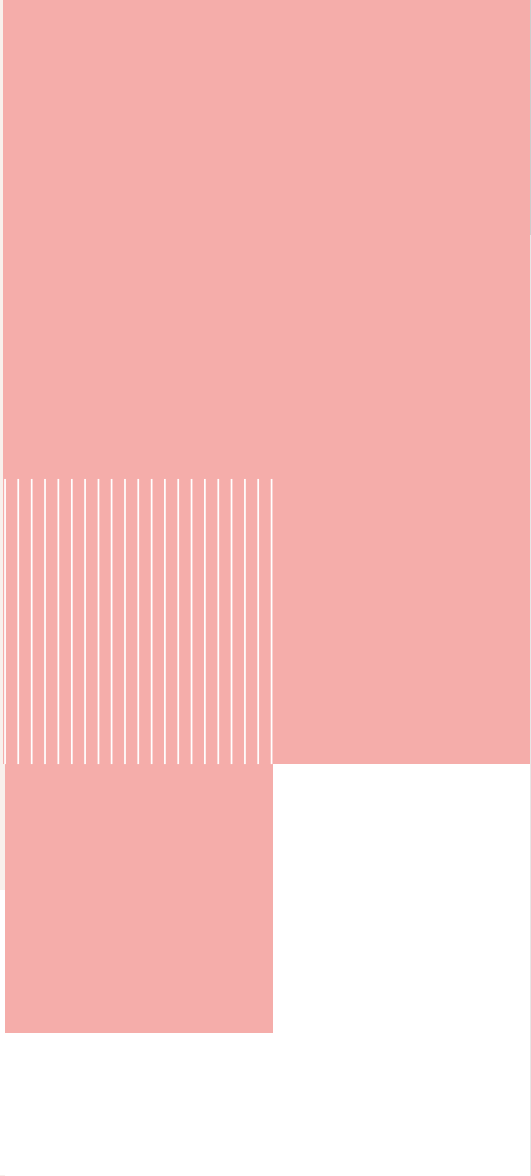
—

- [15] SEAPATH, *yocto-bsp*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/yocto-bsp>
- [16] SEAPATH, *repo-manifest*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/repo-manifest>
- [17] SEAPATH, *meta-seapath*. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://github.com/seapath/meta-seapath>

Bibliographie



Annexes



Annexes

—

Synchronisation de la CI entre dépôts

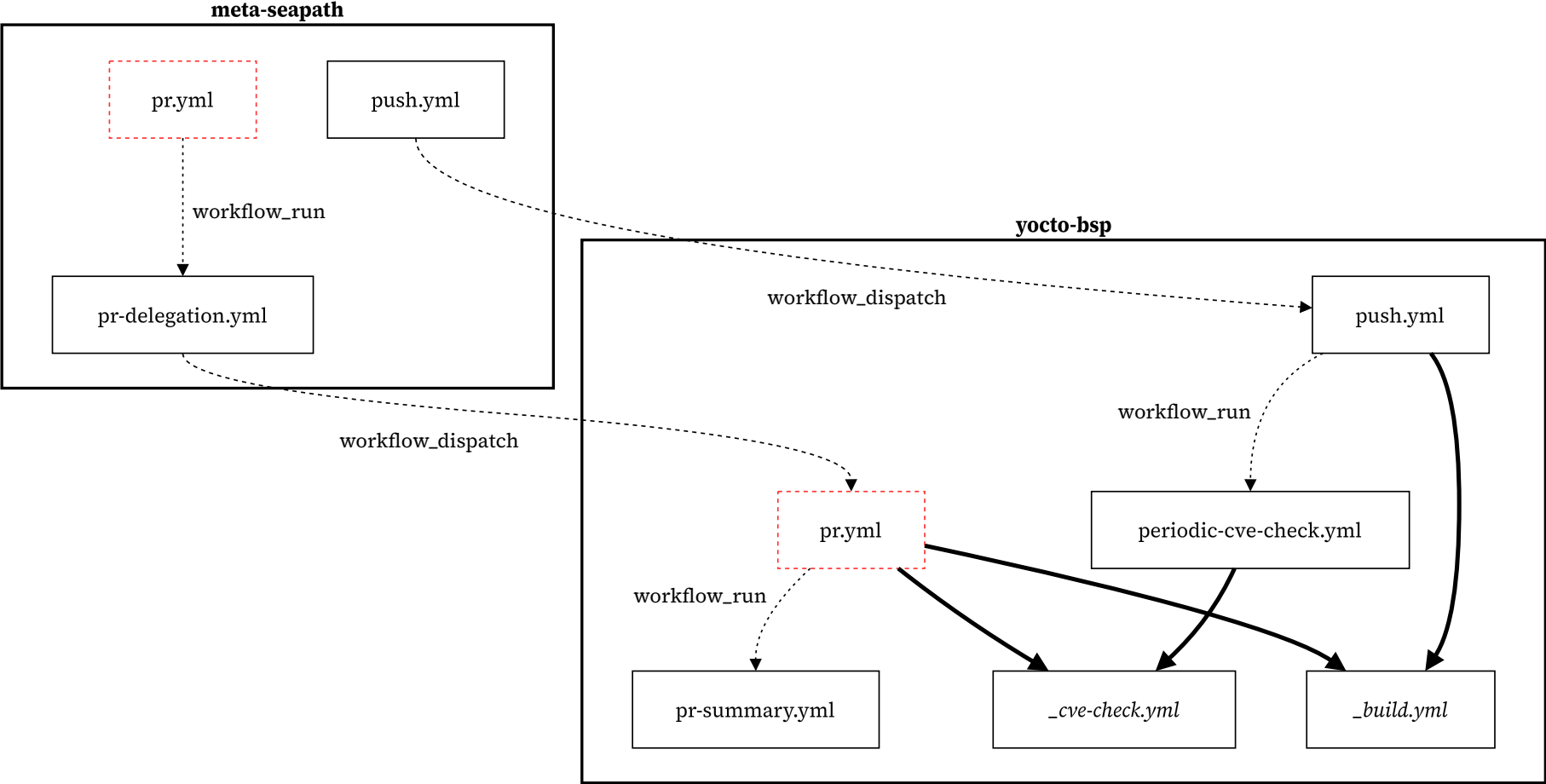


Fig. 51. – Relations entre les pipelines de CI de SEAPATH Yocto

Extrait de fichier SPDX

json

```
1  {
2    "type": "software_package",
3    "software_packageVersion": "20.2.1",
4    "name": "ceph",
5    "summary": "User space components of the Ceph file system"
6    "description": "User space components of the Ceph file system.",
7    "externalIdentifier": [{"identifier": "pkg:yocto/meta-seapath/ceph@20.2.1"}, {"identifier":
8      "cpe:2.3:*:*:ceph_storage_osd:20.2.1:*:*:*:*:*:*"}, ...],
9  },
10 {
11   "type": "software_file",
12   "name": "usr/lib/ceph/erasure-code/libec_clay.so",
13   "verifiedUsing": [{"algorithm": "sha256", "hashValue":
14     "0a323c42c25bf33d08334ac7cd2dda1b10b734f5f07714f9185c6a48db8feee4"}]
15 },
16 {
17   "type": "software_file",
18   "software_primaryPurpose": "source",
19   "name": "sources/linux-mainline-rt-6.12.89-rt18+git/drivers/net/ipa/data/ipa_data-v5.5.c",
20   "verifiedUsing": ...
21 },
```

Annexes
— Extrait de fichier SPDX

VulnScout CI

bash

```
1 for variant in "host_standalone_efi" "guest_efi" ...; do
2   vulnscout \
3     --name "seapath" \
4     --variant "$variant" \
5     --add-spdx "$variant/sbom-cve-checked.spdx.json"
6 done
7
8 vulnscout \
9   --report "summary.adoc" \
10  --report "vulnerabilities.csv"
11
12 vulnscout \
13   --match-condition \
14     "not ignored and not fixed and (cvss >= 9.0 or (cvss >= 7.0 and epss >= 50%))" \
15   --report "failed-vulnerabilities.txt"
```

Annexes

— VulnScout CI

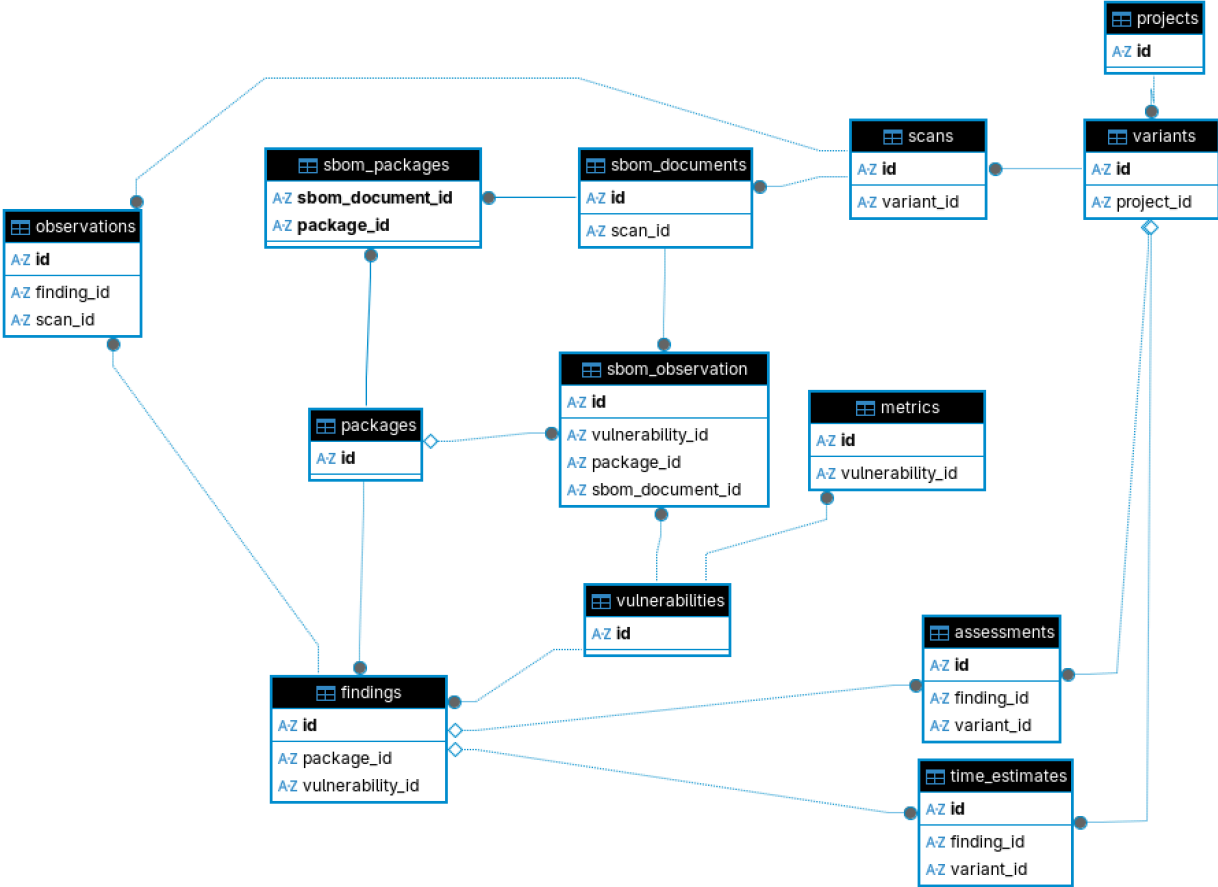
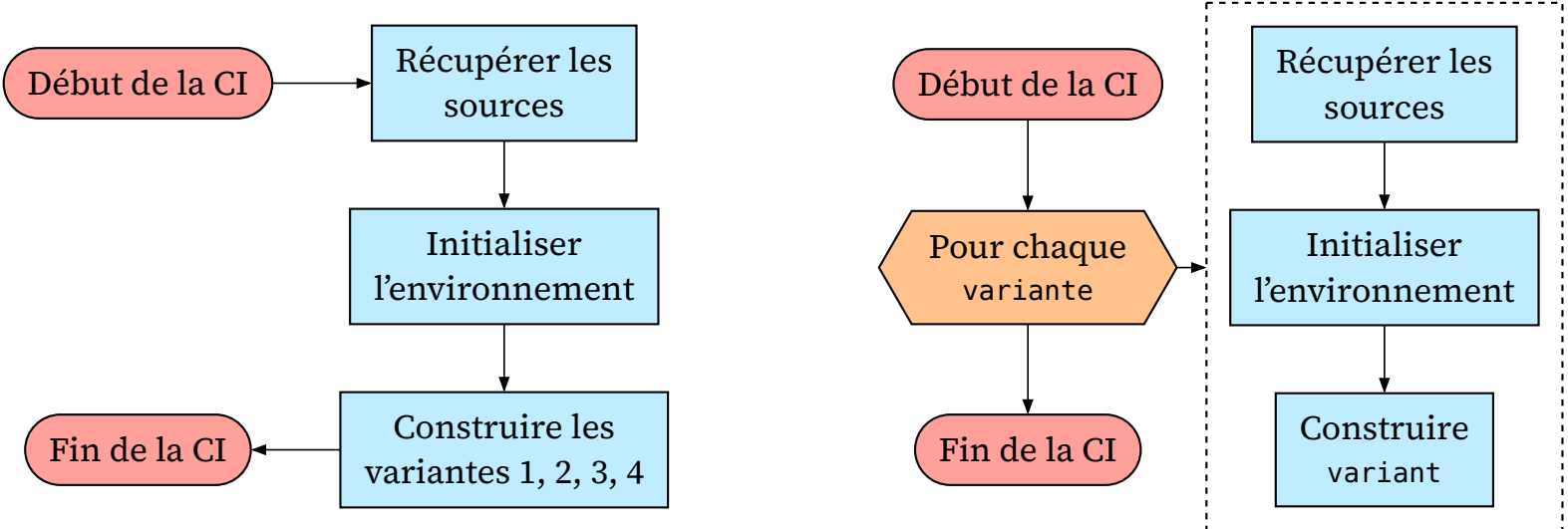


Fig. 52. – Modèle entité-association de la base de données de VulnScout



(a) Pipeline pré-existant

(b) Nouveau pipeline build

Figure 56. – Évolution de l'architecture du pipeline de construction des images

Arborescence de SEAPATH Yocto

```
1 .
2 |— .cqfd
3 |   |— docker
4 |— .github
5 |   |— workflows
6 |— tools
7 |   |— demo_setup
8 |   |— deploy_vm
9 |— > Contenu du dépôt yocto-bsp [15]
10 |   |— .repo
11 |   |— ...
12 |   |— manifests
13 |— > Dépôt repo-manifest [16]
14 |   |— sources
15 |     |— bitbake
16 |     |— meta-yocto
17 |     |— meta-intel
18 |     |— ...
19 |   |— > Dépendances de SEAPATH
20 |     |— meta-seapath
21 |   |— > Dépôt meta-seapath [17]
```

Annexes
— Arborescence de SEAPATH Yocto

```
1 inherit create-spx-2.2
2 inherit vex
3
4 SPDX_INCLUDE_COMPILED_SOURCES:pn-linux-mainline-rt = "1"
```

Fig. 55. – Paramètres pour générer les fichiers nécessaires à la détection de vulnérabilités

```
1 # This allows to get the debug symbols of the kernel and know which files
   have been compiled. It has been pulled from https://git.yoctoproject.org/
   yocto-kernel-cache/tree/features/debug/debug-kernel.cfg?h=yocto-6.12
2 CONFIG_DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT=y
3 CONFIG_DEBUG_INFO=y
```

Fig. 56. – Fichier de configuration rajouté à la recette du noyau Linux